Backpropagation

Last modified: 2026-10-29

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
source("../tools/use-torch.R")
use_torch(c("numpy", "pandas", "matplotlib"))
```

Brian Hutchinson's CSCI 481/581 sets the order of topics in these notes, which follow it with his permission; this page covers the lecture on computing gradients[1] in his module on machine learning[2] fundamentals. The exercises are new. Recording a computation and running it backward follow *Deep Learning*, Section 6.5 [1].

1 Welcome

1.1 Plan for this lecture

Automatic differentiation gave the derivative of a loss without anyone writing it down. This lecture opens it up for the MLP on the digits:

1. how PyTorch records the operations that compute a loss;
2. how the chain rule, applied to those operations from the loss back to the input, gives the gradient[3] with respect to every parameter[4] at once, and how to compute it by hand and match PyTorch;
3. why that costs only a few times as much as computing the loss once;
4. why the gradient can shrink layer by layer in a deep network.

lds's chapter on the same algorithm[5] derives it for a general network with one hidden layer, and runs more experiments on depth[6].

1.2 The digits and the network

```
import time

import numpy as np
import pandas as pd
import torch
from torch import nn
import matplotlib.pyplot as plt

_ = torch.manual_seed(481)
digits = pd.read_csv("data/optdigits-test.csv", header=None)
x = torch.tensor(digits.iloc[:, :64].to_numpy(), dtype=torch.float32) / 16
y = torch.tensor(digits[64].to_numpy())
```

[1] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[2] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-machine-learning
[3] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[4] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[5] https://morrison-lab.github.io/lds/chapters/backpropagation.html
[6] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-depth

```
n_train = 1200
order = torch.randperm(len(x))
x_train, y_train = x[order[:n_train]], y[order[:n_train]]

_ = torch.manual_seed(1)
mlp = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 10))
loss_fn = nn.CrossEntropyLoss()
print(mlp)
```

```
Sequential(
  (0): Linear(in_features=64, out_features=32, bias=True)
  (1): ReLU()
  (2): Linear(in_features=32, out_features=10, bias=True)
)
```

The exercises use the same digits, split and network as the feedforward networks chapter: the 1,797 images of `data/optdigits-test.csv`[7] [2], with pixels divided by 16, a random 1,200 of them as the training set[8], and an untrained MLP with one hidden layer of 32 ReLU units.

2 Recording a computation

2.1 What the loss is made of

Exercise 2.1 (From an image to its loss)**.** List, in order, the operations that take one image $\underset{\sim}{x}$ with true digit y to the MLP's cross-entropy[a] loss.

[a] https://morrison-lab.github.io/pds/expectation.html#def-cross-entropy

Solution

Solution 2.1. Write $\mathbf{W}_1, \underset{\sim}{b}_1$ for the first layer's parameters[a] and $\mathbf{W}_2, \underset{\sim}{b}_2$ for the second's.

1. $\underset{\sim}{z}_1 = \mathbf{W}_1 \underset{\sim}{x} + \underset{\sim}{b}_1$, an affine function[b] of the pixels;
2. $\underset{\sim}{h} = \mathrm{ReLU}(\underset{\sim}{z}_1)$, entry by entry;
3. $\underset{\sim}{z}_2 = \mathbf{W}_2 \underset{\sim}{h} + \underset{\sim}{b}_2$, the logits;
4. $L = -\log \mathrm{softmax}(\underset{\sim}{z}_2)_y$, the cross-entropy[c] loss, where softmax is the softmax function[d].

Each step uses only the input and the results of earlier steps.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[b] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-affine-function
[c] https://morrison-lab.github.io/pds/expectation.html#def-cross-entropy
[d] https://morrison-lab.github.io/lds/chapters/multiclass-logistic-regression.html#def-softmax

Definition 2.1 (Computational graph)**.** A **computational graph** of a calculation has one node for each variable, whether an input, a parameter[a] or an intermediate result, and an arrow from each node to every node computed from it [1, Sec. 6.5.1]. It has no cycles: following the arrows never returns to a node.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Example 2.1 (The graph PyTorch records)**.**

[7] https://morrison-lab.github.io/dds/chapters/data/optdigits-test.csv
[8] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set

```
loss = loss_fn(mlp(x_train[:1]), y_train[:1])
node = loss.grad_fn
recorded = []
while node is not None:
    recorded.append(type(node).__name__)
    # Follow the first input that is not a leaf parameter.
    earlier = [f for f, _ in node.next_functions
               if f is not None and type(f).__name__ != "AccumulateGrad"]
    node = earlier[0] if earlier else None
print(*recorded, sep="\n")
```

```
NllLossBackward0
LogSoftmaxBackward0
AddmmBackward0
ReluBackward0
AddmmBackward0
TBackward0
```

For the first training image, PyTorch's computational graph holds the four steps of Solution 2.1, read from the loss back to the input. Each `grad_fn` names the operation that made a tensor:

- `NllLossBackward0` and `LogSoftmaxBackward0` are step 4, which `nn.CrossEntropyLoss` computes as a log softmax followed by picking out entry y;
- `AddmmBackward0` ("add matrix multiply") is each affine step, 3 and then 1;
- `ReluBackward0` is step 2;
- `TBackward0` is the transpose of $\mathbf{W}_1$ in `x @ weight.T`.

The chain stops at the transpose, since the image itself was not made with `requires_grad=True`.

3 The backward pass

3.1 The gradient at the logits

Exercise 3.1 (The gradient at the logits)**.** Find the derivative of the cross-entropy[a] loss $L = -\log \operatorname{softmax}(\underset{\sim}{z})_y$, where softmax is the softmax function[b], with respect to each logit z_m, for $m = 1, \ldots, C$, where C is the number of classes.

Hint

Start from the log softmax theorem.

[a] https://morrison-lab.github.io/pds/expectation.html#def-cross-entropy
[b] https://morrison-lab.github.io/lds/chapters/multiclass-logistic-regression.html#def-softmax

Solution

Solution 3.1. By the log softmax theorem, $L = \operatorname{LSE}(\underset{\sim}{z}) - z_y$, where LSE is log-sum-exp.

$$
\begin{aligned}
\frac{\partial \operatorname{LSE}(\underset{\sim}{z})}{\partial z_m} &= \frac{\partial}{\partial z_m} \log \sum_{k=1}^{C} \mathrm{e}^{z_k} && \text{(definition of LSE)} \\
&= \frac{1}{\sum_{k=1}^{C} \mathrm{e}^{z_k}} \frac{\partial}{\partial z_m} \sum_{k=1}^{C} \mathrm{e}^{z_k} && \text{(chain rule for log)} \\
&= \frac{1}{\sum_{k=1}^{C} \mathrm{e}^{z_k}} \frac{\partial}{\partial z_m} \mathrm{e}^{z_m} && \text{(only term } k = m \text{ depends on } z_m\text{)} \\
&= \frac{\mathrm{e}^{z_m}}{\sum_{k=1}^{C} \mathrm{e}^{z_k}} && \text{(derivative of the exponential)} \\
&= \operatorname{softmax}(\underset{\sim}{z})_m && \text{(definition of softmax)}
\end{aligned}
$$

The second term, z_y, has derivative 1 with respect to z_y and 0 with respect to every other logit. So

$$
\frac{\partial L}{\partial z_m} = \operatorname{softmax}(\underset{\sim}{z})_m - \mathbb{1}_{m=y}.
$$

Theorem 3.1 (Gradient of the cross-entropy loss at the logits)**.** *For logits $\underset{\sim}{z} \in \mathbb{R}^C$ and true class y, the cross-entropy[a] loss $L = -\log \operatorname{softmax}(\underset{\sim}{z})_y$, where softmax is the softmax function[b], has gradient[c]*

$$
\nabla_{\underset{\sim}{z}} L = \operatorname{softmax}(\underset{\sim}{z}) - \underset{\sim}{e}_y,
$$

where $\underset{\sim}{e}_y$ has 1 in entry y and 0 in every other entry.

[a] https://morrison-lab.github.io/pds/expectation.html#def-cross-entropy
[b] https://morrison-lab.github.io/lds/chapters/multiclass-logistic-regression.html#def-softmax
[c] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

Proof

Proof. By Solution 3.1. □

Example 3.1 (Checking the gradient at the logits)**.**

```
logits = mlp(x_train[:1]).detach().requires_grad_()
loss = loss_fn(logits, y_train[:1])
loss.backward()
by_theorem = torch.softmax(logits, dim=1) - nn.functional.one_hot(y_train[:1], 10)
logit_grad_gap = (logits.grad - by_theorem).abs().max().item()
print(y_train[0].item())
```

```
0
```

```
print(logits.grad.numpy().round(3))
```

```
[[-0.921  0.104  0.125  0.092  0.142  0.082  0.072  0.113  0.106  0.085]]
```

```
print(logit_grad_gap)
```

```
1.4901161193847656e-08
```

For the first training image, a 0, autograd's gradient[a] at the logits matches Theorem 3.1 to within 1.5e-08. Every entry is a predicted probability, near $\frac{1}{10}$, except entry 0, the true digit, which is that probability minus 1.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

3.2 Through a linear layer

Exercise 3.2 (Gradient for a layer's weight matrix)**.** A layer computes $\underset{\sim}{z} = \mathbf{W}\underset{\sim}{h} + \underset{\sim}{b}$, and the gradient[a] $\underset{\sim}{\delta} = \nabla_{\underset{\sim}{z}} L$ of the loss with respect to its output is known. What is the gradient of L with respect to $\mathbf{W}$?

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

Solution

Solution 3.2. Entry j of the output is $z_j = \sum_k W_{jk} h_k + b_j$, so W_{jk} affects L only through z_j.

$$\begin{aligned} \frac{\partial L}{\partial W_{jk}} &= \frac{\partial L}{\partial z_j}\frac{\partial z_j}{\partial W_{jk}} && \text{(chain rule through } z_j\text{)} \\ &= \delta_j h_k && \text{(since } \frac{\partial z_j}{\partial W_{jk}} = h_k\text{)} \end{aligned}$$

As a matrix,

$$\nabla_{\mathbf{W}} L = \underset{\sim}{\delta}\underset{\sim}{h}^{\top},$$

an $m \times n$ outer product for a layer with n inputs and m outputs, with the same shape as $\mathbf{W}$.

Exercise 3.3 (Gradient for a layer's bias)**.** For the layer of Exercise 3.2, what is the gradient[a] of L with respect to $\underset{\sim}{b}$?

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

Solution

Solution 3.3. Bias b_j affects L only through z_j, and $\frac{\partial z_j}{\partial b_j} = 1$, so $\frac{\partial L}{\partial b_j} = \delta_j \cdot 1 = \delta_j$, and

$$\nabla_{\underset{\sim}{b}} L = \underset{\sim}{\delta}.$$

Exercise 3.4 (The gradient passed back)**.** For the layer of Exercise 3.2, find the gradient[a] of L with respect to the layer's input $\underset{\sim}{h}$.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

Solution

Solution 3.4. Input h_k affects every output z_j, so the chain rule adds up a term for each:

$$\begin{aligned} \frac{\partial L}{\partial h_k} &= \sum_j \frac{\partial L}{\partial z_j}\frac{\partial z_j}{\partial h_k} && \text{(chain rule through every } z_j\text{)} \\ &= \sum_j \delta_j W_{jk} && \text{(since } \frac{\partial z_j}{\partial h_k} = W_{jk}\text{)} \\ &= (\mathbf{W}^{\top}\underset{\sim}{\delta})_k && \text{(entry } k \text{ of a matrix-vector product)} \end{aligned}$$

So $\nabla_{\underset{\sim}{h}} L = \mathbf{W}^{\top}\underset{\sim}{\delta}$: the gradient[a] travels back through the layer by the transpose of its weight matrix.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

3.3 Through the ReLU

Exercise 3.5 (Back through the ReLU)**.** The hidden layer is $\underset{\sim}{h} = \text{ReLU}(\underset{\sim}{z}_1)$, entry by entry, and $\nabla_{\underset{\sim}{h}} L$ is known. Find $\nabla_{\underset{\sim}{z}_1} L$.

Solution

Solution 3.5. Entry $h_k = \max(0, z_{1k})$ depends only on z_{1k}, with derivative 1 where $z_{1k} > 0$ and 0 where $z_{1k} < 0$; at $z_{1k} = 0$ PyTorch, and these notes, take it to be 0, as in the slopes of the activation functions. So

$$\frac{\partial L}{\partial z_{1k}} = \frac{\partial L}{\partial h_k} \mathbb{1}_{z_{1k}>0} :$$

the gradient[a] passes through each unit that was positive and stops at each unit that was 0.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

3.4 The whole backward pass

Exercise 3.6 (The backward pass by hand)**.** Computed by hand from Theorem 3.1, Solution 3.2, Solution 3.3, Solution 3.4 and Solution 3.5, without `.backward()`, how closely does the gradient[a] of the MLP's mean loss over the training set[b] with respect to $\mathbf{W}_1$, $\underset{\sim}{b}_1$, $\mathbf{W}_2$ and $\underset{\sim}{b}_2$ match the one `loss.backward()` gives?

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set

Solution

Solution 3.6.

```
first, second = mlp[0], mlp[2]

# Forward, keeping every intermediate result.
z1 = x_train @ first.weight.T + first.bias
h = torch.relu(z1)
z2 = h @ second.weight.T + second.bias
loss = loss_fn(z2, y_train)

mlp.zero_grad()
loss.backward()

with torch.no_grad():
    # Backward, from the loss to the first layer, one row per image.
    g2 = (torch.softmax(z2, dim=1) - nn.functional.one_hot(y_train, 10)) / n_train
    grad_W2 = g2.T @ h
    grad_b2 = g2.sum(dim=0)
    grad_h = g2 @ second.weight
    grad_z1 = grad_h * (z1 > 0)
    grad_W1 = grad_z1.T @ x_train
    grad_b1 = grad_z1.sum(dim=0)

pairs = [(grad_W1, first.weight), (grad_b1, first.bias),
         (grad_W2, second.weight), (grad_b2, second.bias)]
backward_gap = max((mine - p.grad).abs().max().item() for mine, p in pairs)
print(backward_gap)
```

```
3.725290298461914e-09
```

The four gradients[a] match autograd's to within 3.7e-09. The code runs the steps of Solution 2.1 forward, then their derivatives in reverse:

- `g2` is $\nabla_{\underset{\sim}{z}_2} L$ from Theorem 3.1, for every image at once, divided by 1200 because the loss is the mean over that many images;
- `grad_W2` and `grad_b2` are Solution 3.2 and Solution 3.3, with the outer products of the images added up by one matrix product;
- `grad_h` is Solution 3.4, written for rows instead of columns;
- `grad_z1` is Solution 3.5;
- `grad_W1` and `grad_b1` are Solution 3.2 and Solution 3.3 again, for the first layer.

The backward steps reuse `z1` and `h` from the forward pass.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

Definition 3.1 (Backpropagation (backprop))**. Backpropagation**, or **backprop**, computes the gradient[a] of a scalar, such as a loss, with respect to every node of a computational graph: it visits the nodes in the reverse of the order the forward computation made them, and at each node applies the chain rule, combining the gradient with respect to the node's output with the derivative of the node's own operation [1, Sec. 6.5], [3].

Backpropagation is reverse-mode automatic differentiation applied to a network's loss, and PyTorch's `.backward()` runs it. Some sources use "backpropagation" for the whole training algorithm, including the parameter[b] update; these notes use it only for computing the gradient.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Example 3.2 (Backpropagation through the MLP)**.** Solution 3.6 is backpropagation through the four steps of Solution 2.1. It visits them in reverse order: the loss gives the gradient[a] at the logits `g2`; the second affine step gives the gradients[b] of $\mathbf{W}_2$, $\underset{\sim}{b}_2$ and the hidden layer; the ReLU gives the gradient at

$\underset{\sim}{z}_1$; and the first affine step gives the gradients of $\mathbf{W}_1$ and $\underset{\sim}{b}_1$.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[b] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

4 Cost

4.1 Backpropagation against finite differences

Exercise 4.1 (The cost of finite differences)**.** A finite difference, $(L(\theta_i + \varepsilon) - L(\theta_i))/\varepsilon$ for a small ε, estimates the derivative of the MLP's mean training loss L with respect to one parameter[a] θ_i, with every other parameter held fixed. How many evaluations of the loss does that take to estimate the gradient[b] with respect to P parameters[c]?

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[b] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Solution

Solution 4.1. $P + 1$: one at the current parameters[a], and one more for each parameter[b] nudged. Backpropagation gets all P derivatives from one forward pass and one backward pass, and each backward step costs a small multiple of its forward step: a matrix product forward becomes two matrix products backward, as in Solution 3.2 and Solution 3.4, so the two passes together cost about three forward passes.

```
def mean_loss():
    return loss_fn(mlp(x_train), y_train)

start = time.perf_counter()
for repeat in range(20):
    mlp.zero_grad()
    mean_loss().backward()
backprop_seconds = (time.perf_counter() - start) / 20

start = time.perf_counter()
with torch.no_grad():
    base = mean_loss()
    w = second.weight
    estimate = torch.zeros_like(w)
    for j in range(w.shape[0]):
        for k in range(w.shape[1]):
            w[j, k] += 1e-3
            estimate[j, k] = (mean_loss() - base) / 1e-3
            w[j, k] -= 1e-3
difference_seconds = time.perf_counter() - start
fd_gap = (estimate - second.weight.grad).abs().max().item()
print(backprop_seconds, difference_seconds, fd_gap)
```

```
0.0012992372000013575 0.30171424200000274 0.000441218726336956
```

One forward and backward pass over the training set[c] took 1.30 milliseconds, for every parameter of the MLP. Finite differences for only the 320 entries of $\mathbf{W}_2$ took 302 milliseconds, 232 times as long, and agree with backpropagation only to within 0.00044, since a finite difference is an approximation. Timings vary from run to run; the ratio grows with the number of parameters[d].

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[d] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Remark (What backpropagation stores)

Remark 4.1 (What backpropagation stores). The backward steps of Solution 3.6 reuse `z1` and `h` from the forward pass, so PyTorch keeps every intermediate result that a later backward step needs until `.backward()` has run. That memory grows with the number of layers and with the batch size, and it is often what limits the batch size on a GPU. Code run inside `torch.no_grad()`, such as computing test accuracy[a], records nothing and keeps nothing.

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy

5 Gradients in deep networks

5.1 Gradient sizes layer by layer

Exercise 5.1 (Gradients in a deep network)**.** Take two MLPs for the digits with 10 hidden layers of 32 units each, one with the logistic function[a] as its activation function and one with the ReLU. At their starting weights, how does the size of the gradient[b] of the mean training loss with respect to each layer's weight matrix change from the last layer to the first?

[a] https://morrison-lab.github.io/mcs/functions.html#def-logistic-function
[b] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

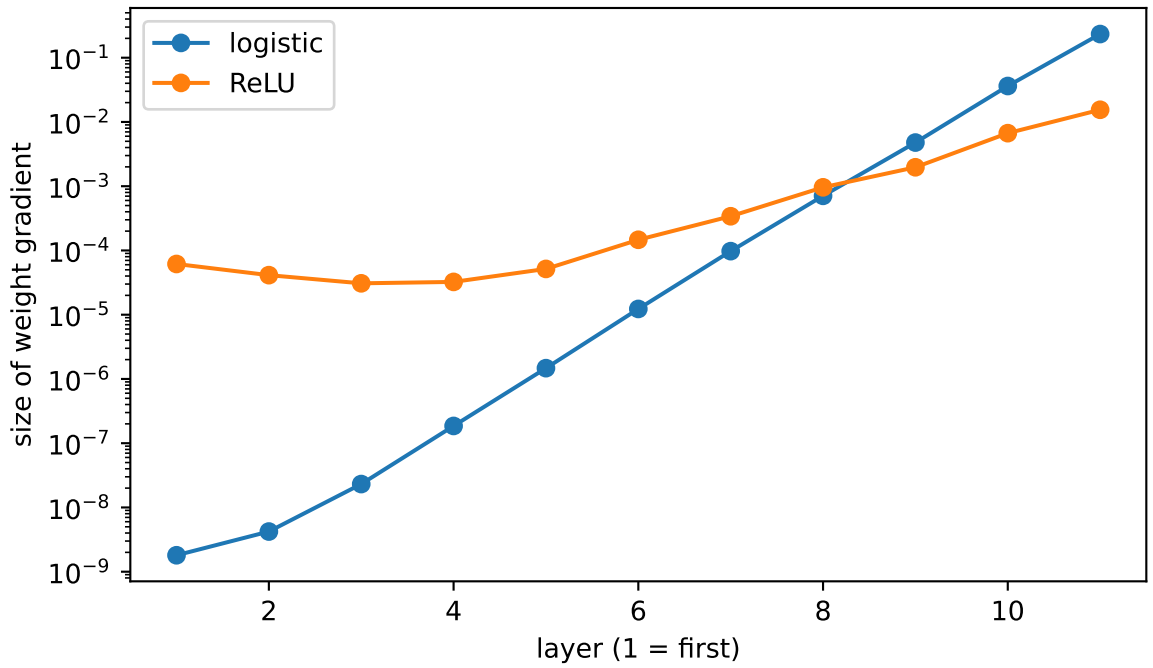
Solution

Solution 5.1.

```
def deep_mlp(activation, n_hidden=10, width=32):
    _ = torch.manual_seed(0)
    layers, n_in = [], 64
    for layer in range(n_hidden):
        layers += [nn.Linear(n_in, width), activation()]
        n_in = width
    layers.append(nn.Linear(width, 10))
    return nn.Sequential(*layers)


grad_norms = {}
for name, activation in [("logistic", nn.Sigmoid), ("ReLU", nn.ReLU)]:
    net = deep_mlp(activation)
    loss_fn(net(x_train), y_train).backward()
    grad_norms[name] = [m.weight.grad.norm().item() for m in net if isinstance(m, nn.Linear)]

fig, ax = plt.subplots(figsize=(6, 3.5), layout="constrained")
for name, norms in grad_norms.items():
    ax.semilogy(range(1, len(norms) + 1), norms, marker="o", label=name);
ax.set_xlabel("layer (1 = first)");
ax.set_ylabel("size of weight gradient");
ax.legend();
plt.show()
```




```
shrink = {name: norms[-1] / norms[0] for name, norms in grad_norms.items()}
print(shrink)
```

```
{'logistic': 129510924.3979597, 'ReLU': 251.30356569772417}
```

The size of a matrix here is the square root of the sum of its squared entries. In both networks the weight gradient[a] is largest at the last layer. With the logistic function[b], the weight gradient shrinks at every layer going back, and the last layer's weight gradient is about 10^8 times the first layer's. With the ReLU, the weight gradient shrinks over the last few layers and then levels off, and the last layer's is 251 times the first layer's. A training step with one learning rate[c] would move the logistic network's first layers almost not at all.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[b] https://morrison-lab.github.io/mcs/functions.html#def-logistic-function
[c] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-learning-rate

5.2 Why they shrink

Exercise 5.2 (Why the gradient shrinks)**.** In a deep MLP, hidden layer ℓ is $\underset{\sim}{h}_\ell = g(\underset{\sim}{z}_\ell)$ with $\underset{\sim}{z}_\ell = \mathbf{W}_\ell \underset{\sim}{h}_{\ell-1} + \underset{\sim}{b}_\ell$. Why does the gradient[a] shrink from layer to layer, going back, when g is the logistic function[b]?

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[b] https://morrison-lab.github.io/mcs/functions.html#def-logistic-function

Solution

Solution 5.2. Applying Solution 3.5's reasoning to g, then Solution 3.4, takes the gradient[a] back one layer:

$$\nabla_{\underset{\sim}{h}_{\ell-1}} L = \mathbf{W}_\ell{}^\top \underset{\sim}{\delta}_\ell, \qquad \underset{\sim}{\delta}_\ell = \nabla_{\underset{\sim}{z}_\ell} L = g'(\underset{\sim}{z}_\ell) \odot \nabla_{\underset{\sim}{h}_\ell} L,$$

where $\odot$ multiplies entry by entry. Each layer back multiplies the gradient by the slopes $g'(\underset{\sim}{z}_\ell)$ and by $\mathbf{W}_\ell{}^\top$. The logistic function[b]'s slope is at most $\frac{1}{4}$ (lds's bound on it[c]), and less where units saturate[d], so unless the weights are large, every layer shrinks the gradient by a factor less than 1, and 10 layers multiply 10 such factors. The ReLU's slope has no $\frac{1}{4}$ bound: it is 1 at each unit that is positive and 0

at each unit that is not. In the ReLU network of Solution 5.1, the gradient going back still shrinks, both because the weights PyTorch starts with are small and because the units at 0 pass none of it back. The weight gradient of layer ℓ is $\underset{\sim}{\delta}_\ell \underset{\sim}{h}_{\ell-1}{}^\top$, by Solution 3.2, so its size depends on the layer's input $\underset{\sim}{h}_{\ell-1}$ as well as on the gradient $\underset{\sim}{\delta}_\ell$ at its output. In the ReLU network the layer inputs grow toward the first layers, which is why its weight gradients[e] level off there while the gradients $\underset{\sim}{\delta}_\ell$ keep shrinking.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[b] https://morrison-lab.github.io/mcs/functions.html#def-logistic-function
[c] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#thm-logistic-derivative-bound
[d] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-saturation
[e] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

Definition 5.1 (Vanishing gradient)**.** A network has a **vanishing gradient** when the gradient[a] of its loss with respect to the parameters[b] of its early layers is many orders of magnitude smaller than with respect to those of its later layers, because backpropagation multiplies it by a factor smaller than 1 at each layer [1, Sec. 8.2.5], [4]. The early layers then learn far more slowly than the later ones, or not at all.
The opposite, a gradient that grows layer by layer, is an exploding gradient[c].

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[c] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-exploding-gradient

Example 5.1 (The deep logistic network)**.** The 10-hidden-layer logistic network of Solution 5.1 has a vanishing gradient: its first layer's weight gradient[a] is about 10^{-8} times its last layer's.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

Remark (What keeps gradients from vanishing)

Remark 5.1 (What keeps gradients from vanishing). The ReLU removes the $\frac{1}{4}$ bound on each layer's slopes. Other remedies are starting the weights at a scale that keeps the gradient[a]'s size steady from layer to layer, such as He initialization[b] or Glorot initialization[c], normalizing each layer's values, and residual connections[d], which give the gradient a path around each layer. lds's experiment on depth[e] measures the same effect.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[b] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-he-initialization
[c] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-glorot-initialization
[d] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-residual-connection
[e] https://morrison-lab.github.io/lds/chapters/backpropagation.html#sec-vanishing-experiment

5.3 Further reading

Read

Deep Learning's section 6.5[a] covers backpropagation on general computational graphs [1, Sec. 6.5].

[a] https://www.deeplearningbook.org/contents/mlp.html

6 References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org/

[2] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits." UCI Machine Learning Repository, 1998. doi: 10.24432/C50P49[9].

[3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986, doi: 10.1038/323533a0[10].

[4] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, in Proceedings of machine learning research, vol. 9. PMLR, 2010, pp. 249–256. Available: https://proceedings.mlr.press/v9/glorot10a.html

[9] https://doi.org/10.24432/C50P49
[10] https://doi.org/10.1038/323533a0